One Counter per Resource Family: Leakage-Aware Evaluation of Compact PMU Phase Prediction

Keshav Krishna
Department of Computer Science
New York University
keshaviitropar@gmail.com

Abstract

Machine learning can turn hardware performance counters into online workload-phase predictors, but standard evaluation choices can make a predictor appear much more deployable than it is. Randomly splitting correlated intervals leaks executions, repeating the current offline phase creates an oracle baseline, overall accuracy hides missed transitions, and model parameters omit counter discretization and history state. We present FAMILYPHASE, a compact and auditable interface in which a train-only full-counter teacher discovers a global resource-pressure label while a deployable student observes one validation-selected counter state per available resource family. The evaluation groups complete co-running executions, reports transition-event metrics and group-bootstrap uncertainty, distinguishes oracle from deployable inputs, and budgets the complete online state. In an audited 60-run PARSEC pilot, a depth-six tree over five 20-interval histories reaches 98.21% held-out accuracy using an estimated 201.25 bytes. Against a matched current-state-only tree, history improves accuracy by 0.60 percentage points (paired 95% CI: 0.29–0.92) and transition-event F1 from 28.6% to 43.9%. The audit also exposes small transition counts, unstable three-state clustering, and poor PMU enabled time. The main result is therefore an evaluation protocol and reproducible study design, not an uncalibrated energy claim.

1 Why phase-prediction accuracy is easy to overstate

Hardware performance-monitoring units (PMUs) offer direct signals about cache, memory, branch, and execution behavior. Learned phase predictors can use these signals for scheduling, cache adaptation, or performance control [Sherwood et al., 2003, Perelman et al., 2006, Kim et al., 2017, Lozano and Gerstlauer, 2022]. A practical predictor, however, must work with few physical counter slots and a small online state. It must also predict a label that is not already available at deployment.

Four common shortcuts obscure those requirements. First, a random interval split puts near-identical samples from one execution in training and test. In multicore data it can also separate two processes that ran together. Second, persistence and Markov baselines are often given the offline teacher's true current phase. Such baselines measure label persistence but cannot be implemented without a separate online phase estimator. Third, stable intervals dominate: a model can achieve high accuracy while never identifying a phase change. Finally, reporting only tree or network parameters omits the state needed to discretize counters and store history.

We study the following deliberately constrained question:

> Can one counter state from each resource family predict the next global phase, and can that claim survive execution-disjoint evaluation, deployable baselines, transition metrics, PMU-quality auditing, and a complete state budget?

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

Our contribution is methodological. We combine (1) a train-only teacher/student interface with validation-only counter selection, (2) leakage-aware splitting and execution-group bootstrap inference, and (3) an information-set audit that prevents oracle phase labels from being presented as online inputs. An accompanying collector enumerates all workload pairs, records exact event encodings and enabled time, and emits the tables needed to replace the preliminary evidence below.

2 Relation to prior phase learning

Program-phase methods traditionally summarize basic-block execution and cluster the resulting vectors offline [Sherwood et al., 2001, 2002]. Subsequent work compared phase representations, predicted recurring phases, and extended phase analysis to parallel programs [Dhodapkar and Smith, 2003, Sherwood et al., 2003, Perelman et al., 2006]. More recent systems use hardware counters and learned models to make prediction inexpensive enough for online control [Nomani and Szefer, 2015, Kim et al., 2017, Lozano and Gerstlauer, 2022]. FAMILYPHASE is complementary: its novelty is not another high-capacity predictor, but an evaluation boundary around a deliberately small one-counter-per-family student. That boundary matters because physical event slots are scarce, multiplexed events can distort labels, and discretization and history consume state even when model parameters are tiny [Eyerman et al., 2006]. We therefore ask whether the compact interface survives execution-disjoint and workload-disjoint tests before connecting it to a resource controller.

3 A compact, explicit information interface

Global teacher. For run r and interval t, let $x_{r,t} \in \mathbb{R}^d$ contain the safe nonnegative PMU counts available on the host. Timing-derived quantities such as IPC, CPI, elapsed time, and requested frequency are excluded so that a future controller does not feed its own action back into the label. Training medians fill missing values; $\log(1+x)$ and training-only standardization limit count skew. The offline teacher fits k-means on training rows,

$$c_{r,t} = \arg\min_j \|\widetilde{x}_{r,t} - \mu_j\|_2^2, \tag{1}$$

and freezes its preprocessing and centroids for validation and test. Cluster IDs are ordered by a resource-pressure score for interpretation. They are descriptive regions of the observed counter space, not ground-truth bottleneck or power labels.

Reduced student input. Candidate events are assigned to L1, L2, LLC, branch/control, core/FP, and memory/off-core families. A depth-limited singleton tree is trained for each supported event, and validation accuracy plus high-pressure recall selects one event $e_f^\star$ per family. Test rows never select counters. A train-only one-dimensional three-cluster model converts each selected count into $s_{f,t} \in \{0,1,2\}$. The deployable history model predicts

$$h([s_{1,t-H+1:t}, \ldots, s_{F,t-H+1:t}]) \to c_{r,t+1} \tag{2}$$

with a shallow CART tree [Breiman et al., 1984]. All family streams share the same global target; a family name identifies an input, not a separate phase ground truth.

Information-set audit. Table 1 separates diagnostic and deployable models. Persistence, state-conditioned majority, Markov, run-length Markov, and duration models consume the true offline $c_{r,t}$ and are labeled oracle diagnostics. The fair history ablation is a tree with identical depth and leaf constraints that observes only the current vector $s_{1..F,t}$. A training-prior majority model is a second deployable sanity check.

Complete analytical state budget. For a tree with I internal nodes, L leaves, D inputs, and $N = I + L$ nodes, we estimate

$$B_{\text{tree}} = \frac{I(\lceil \log_2 D \rceil + 16 + 2\lceil \log_2 N \rceil) + 2L}{8}. \tag{3}$$

The online total adds $\lceil 2FH/8 \rceil$ bytes for two-bit history and $4F$ bytes for two 16-bit discretizer thresholds per family. This is storage, not synthesized area or energy.

Table 1: Inputs available at prediction time. "Teacher phase" means the offline full-counter label and is not a deployable input.

Model	Prediction-time input	Deployable?
Majority	training label prior	yes
Persistence/Markov	true teacher $c_{r,t}$	no: oracle
Current-state tree	selected $s_{1..F,t}$	yes
History tree	selected $s_{1..F,t-H+1:t}$	yes

Table 2: Pilot configuration-group holdout. Accuracy CIs resample nine test executions. Transition F1 treats a label change as an event.

Model	Accuracy (95% CI)	Macro F1	Trans. acc.	Trans. F1
Majority	77.03 (70.77–82.69)	29.01	48.4	9.4
Oracle persistence	97.69 (97.20–98.14)	95.97	0.0	0.0
Current-state tree	97.61 (96.84–98.30)	95.45	12.9	28.6
History tree	**98.21** (97.64–98.71)	**96.88**	**25.8**	**43.9**

4 Evaluation protocol

Leakage control. The split unit is a complete execution scenario. All intervals and repetitions from a configuration remain together; for co-runners, every process in the concurrent group receives the same split. Leave-one-workload-out evaluation moves every scenario containing the held-out workload to test, including its partners. Counter selection uses validation data only. Preprocessing, teacher centroids, counter discretizers, and student models use training data only.

Metrics and uncertainty. We report overall accuracy, macro F1, balanced accuracy, high-pressure recall, and separate stable- and transition-destination accuracy. Treating a predicted label change as an event gives transition precision, recall, F1, and false-alarm rate. The 95% intervals resample complete execution/concurrent-group IDs rather than intervals. History versus current-state comparisons use paired resampling of the same groups. Clustering sensitivity covers $k \in \{2, 3, 4, 5\}$ and five seeds using adjusted Rand index (ARI), train distance, centroid separation, and minimum cluster share.

Audited pilot. The reproducible pilot contains 60 executions of five PARSEC workloads [Bienia et al., 2008], four thread counts, and three repetitions per configuration. It was collected at 10 ms on a dual-socket Intel Xeon E5-2680 v2 host using Linux `perf` 5.14. The merged table contains 9,409 intervals and eight usable counters; L2 was unavailable, leaving five online families. The primary grouped split contains 5,642 training and 1,341 test history windows, but only nine test execution groups and 31 true transitions. We therefore call every number below preliminary.

5 What the pilot establishes—and what it does not

Table 2 shows why the information audit matters. Oracle persistence reaches 97.69% accuracy because the target rarely changes, yet it correctly predicts no transition destination. The deployable current-state tree reaches 97.61% accuracy and 28.6% transition-event F1. Adding 20 intervals of history reaches 98.21% accuracy and 43.9% event F1. The paired accuracy improvement is 0.60 points (95% CI: 0.29–0.92). Transition-destination accuracy improves by 12.95 points, but its interval is 0.0–26.92; the pilot is underpowered for a strong transition claim.

Workload-disjoint evaluation is harder. Averaged descriptively over five leave-one-workload-out folds, the history tree reaches 94.60% accuracy and 43.9% transition-destination accuracy, versus 93.01% and 22.5% for the current-state tree. The bodytrack holdout falls to 82.24% history accuracy, demonstrating that the configuration split alone is insufficient. Accuracy improves reliably in four folds; canneal's 0.07-point difference has a paired 95% CI of -0.38–0.63.

Three audit findings prevent a broader claim. First, $k = 2$ is almost invariant across seeds (mean ARI 0.9997), while $k = 3$ has mean ARI 0.927 and a smallest cluster of only 2.1% of training rows. Low/moderate/high semantics do not by themselves justify the less stable choice. Second, the eight

Table 3: Pilot leave-one-workload-out results (%). Transition accuracy is destination accuracy on true changes. Means are descriptive across five folds.

Held out	Accuracy		Trans. accuracy	
	Current	History	Current	History
blackscholes	97.68	98.24	21.43	40.48
bodytrack	76.47	82.24	21.04	47.26
canneal	99.06	98.99	17.65	35.29
fluidanimate	98.38	99.15	12.50	37.50
freqmine	93.45	94.40	40.00	59.09
Mean	93.01	94.60	22.52	43.93

teacher events average 90.6–91.0% enabled time, but their fifth percentiles are only 64–66%, and roughly one-third of samples fall below 90%. Multiplexing quality must accompany model accuracy. Third, 31 test transitions cannot support an end-to-end control conclusion.

The complete online-state estimate is 64.88 bytes for the current-state tree and 201.25 bytes for the history tree. The latter includes 25 bytes of history and 20 bytes of thresholds. These values show compactness but not RTL area, latency, or energy. We intentionally remove an earlier proxy that mapped phase labels to arbitrary cache-power factors: no cache capacity, miss latency, energy, or slowdown model had been calibrated.

6 Real-data study and ML-for-systems lessons

The accompanying one-command study targets three regimes: a single multithreaded process, two single-threaded co-runners, and two four-thread co-runners. Seven PARSEC workloads produce all $\binom{7}{2} = 21$ cross-workload pairs, with ten repetitions, fixed CPU affinity, randomized execution-group order, exact platform/event manifests, and PMU enabled-time summaries. It runs configuration-group and leave-one-workload-out evaluation together, performs k/seed sensitivity, and emits paired current-versus- history bootstraps. Large data, environments, and caches are confined to scratch storage.

This design yields three lessons for ML-for-systems evaluation. **Information sets are part of the model**: a simple Markov method with an unavailable state is not a deployable baseline. **Correlation defines the split unit**: systems traces are not independent examples, and co-runners cannot be separated. **Measurement quality is model quality**: an offline teacher trained on poorly scheduled events can produce a precise student of an unstable target. These checks precede architecture search; a larger neural model cannot repair leakage or unavailable inputs.

The planned evidence does not guarantee cross-platform portability. PMU semantics, event availability, workload mix, NUMA placement, and operating-system activity remain host dependent. A later archival paper should add a second contemporary platform and, if it claims control benefit, an actual actuator such as cache allocation or DVFS with measured performance and energy. The present claim is intentionally narrower: a compact phase interface and a protocol for evaluating it without granting the learner information it will not have online.

7 Conclusion

Compact PMU histories can predict a persistent offline phase label, but accuracy alone does not establish a useful online system. FAMILYPHASE makes the target, prediction-time inputs, split unit, transition behavior, PMU quality, and full state budget explicit. The pilot finds a small paired benefit from history and, more importantly, reveals the conditions under which that result may fail. We release the corrected study workflow so the final claim can be decided by workload-disjoint multicore evidence rather than interval leakage or an oracle baseline.

References

Christian Bienia, Sanjeev Kumar, Jaswinder Pal Singh, and Kai Li. The parsec benchmark suite: Characterization and architectural implications. In *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques*, pages 72–81, 2008. doi: 10.1145/1454115.1454128.

Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth, 1984.

Ashutosh S. Dhodapkar and James E. Smith. Comparing program phase detection techniques. In *Proceedings of the 36th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 217–227, 2003. doi: 10.1109/MICRO.2003.1253197.

Stijn Eyerman, Lieven Eeckhout, Tejas Karkhanis, and James E. Smith. A performance counter architecture for computing accurate cpi components. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 175–184, 2006. doi: 10.1145/1168857.1168880.

Yeseong Kim, Pietro Mercati, Ankit More, Emily Shriver, and Tajana Rosing. p^4: Phase-based power/performance prediction of heterogeneous systems via neural networks. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*, pages 683–690, 2017. doi: 10.1109/ICCAD.2017.8203843.

Erika Susana Alcorta Lozano and Andreas Gerstlauer. Learning-based phase-aware multi-core cpu workload forecasting. *ACM Transactions on Design Automation of Electronic Systems*, 28(2), 2022. doi: 10.1145/3564929.

Junaid Nomani and Jakub Szefer. Predicting program phases and defending against side-channel attacks using hardware performance counters. In *Proceedings of the Fourth Workshop on Hardware and Architectural Support for Security and Privacy*, 2015. doi: 10.1145/2768566.2768575.

Erez Perelman, Marzia Polito, Jean-Yves Bouguet, John Sampson, Brad Calder, and Carole Dulong. Detecting phases in parallel applications on shared memory architectures. In *Proceedings of the 20th IEEE International Parallel and Distributed Processing Symposium*, 2006. doi: 10.1109/IPDPS.2006.1639325.

Timothy Sherwood, Erez Perelman, and Brad Calder. Basic block distribution analysis to find periodic behavior and simulation points in applications. In *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques*, pages 3–14, 2001. doi: 10.1109/PACT.2001.953283.

Timothy Sherwood, Erez Perelman, Greg Hamerly, and Brad Calder. Automatically characterizing large scale program behavior. In *Proceedings of the 10th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 45–57, 2002. doi: 10.1145/605397.605403.

Timothy Sherwood, Suleyman Sair, and Brad Calder. Phase tracking and prediction. In *Proceedings of the 30th International Symposium on Computer Architecture*, pages 336–349, 2003. doi: 10.1145/859618.859657.